Whiteboard-First Teaching Methodology: The Visual Architecture Playbook

1. Why Whiteboard-First Dominates Slides

At GLA University, students see hundreds of generic PowerPoint slides every semester. Slide reading creates immediate cognitive fatigue and signals that the instructor is merely reciting third-party material. When you pick up a whiteboard marker, three psychological transformations occur:

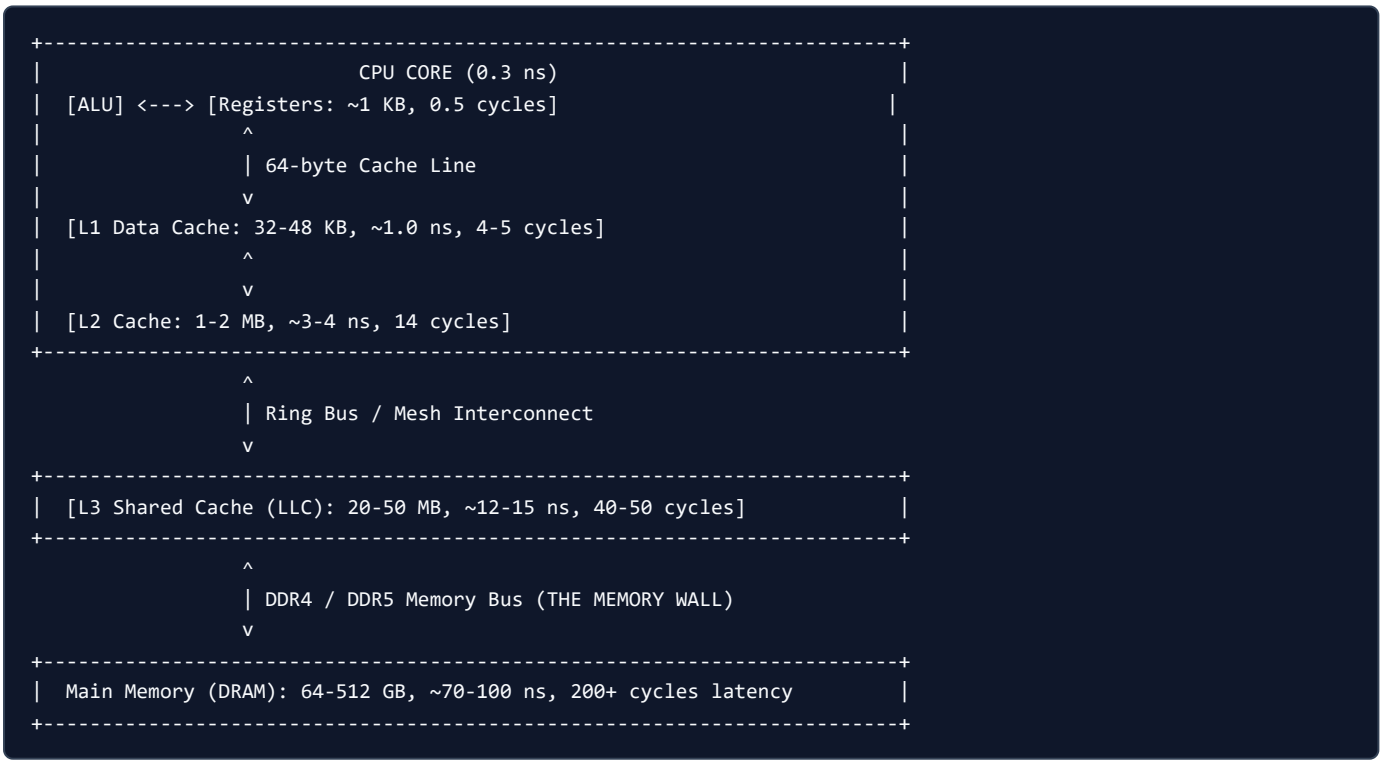
1. **Perception of Mastery**: Students intuitively understand that an instructor who can draw silicon architectures, memory layouts, and execution pipelines from scratch on a blank board possesses genuine, unscripted mastery.
2. **Pacing and Retention**: Drawing forces a natural, digestible pedagogical cadence. Students can trace the flow of data step-by-step rather than being overwhelmed by a dense bullet-point slide.
3. **Active Focus**: Every eye tracks the marker.

---

2. Master Whiteboard Schematics (Day-by-Day Blueprints)

Day 1: The Memory Wall & Vector Silicon Architecture

Schematic 1.1: The Von Neumann Bottleneck vs Modern Memory Hierarchy



Podium Talking Point: *"Notice the discrepancy: the ALU computes in 0.3 nanoseconds, but fetching a floating-point number from DRAM takes 80 nanoseconds! That is 250 wasted CPU clock cycles doing absolutely nothing (a CPU stall). HPC is not about writing faster math; HPC is about keeping the ALU fed with data so it never stalls on the memory bus!"*

Schematic 1.2: Scalar Execution vs SIMD (AVX-512 / Vectorization)

```
SCALAR EXECUTION (Traditional x86):
Cycle 1:  [ a[0] ] + [ b[0] ] = [ c[0] ]
Cycle 2:  [ a[1] ] + [ b[1] ] = [ c[1] ]
Cycle 3:  [ a[2] ] + [ b[2] ] = [ c[2] ]
Cycle 4:  [ a[3] ] + [ b[3] ] = [ c[3] ]  --> 4 instructions, 4 cycles

SIMD VECTOR EXECUTION (AVX-512 / 512-bit Register ZMM0):
Cycle 1:
ZMM0: [ a[0] | a[1] | a[2] | a[3] | a[4] | ... | a[15] ] (16 x 32-bit floats)
  +
ZMM1: [ b[0] | b[1] | b[2] | b[3] | b[4] | ... | b[15] ]
  =
ZMM2: [ c[0] | c[1] | c[2] | c[3] | c[4] | ... | c[15] ] --> 1 instruction, 1 cycle!
```

Podium Talking Point: *"In one single clock cycle, Intel AVX-512 adds sixteen 32-bit floating-point numbers simultaneously. If your compiler is not emitting vector instructions, you are utilizing exactly 1/16th (6.25%) of your hardware's theoretical compute capacity!"*

Day 2: Shared Memory (OpenMP) vs Distributed Memory (MPI)

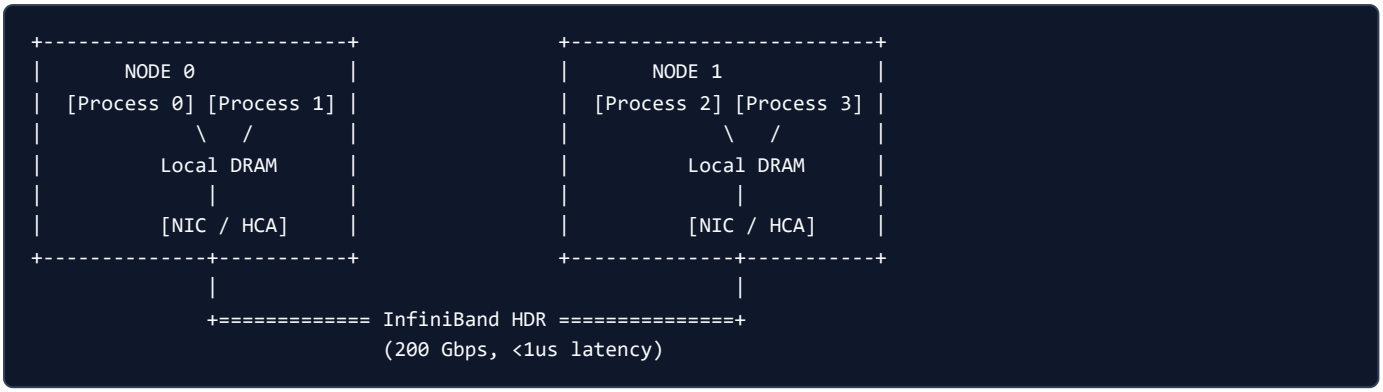
Schematic 2.1: OpenMP Fork-Join & The False Sharing Trap

```
FALSE SHARING PHENOMENON (The Silent Performance Killer):

          One 64-byte Cache Line in L1/L2 Cache
+-------------------------------------------------------------+
|  sum[0] (Thread 0)  |  sum[1] (Thread 1)  |  sum[2]  | sum[3] |
+-------------------------------------------------------------+
         |                      |
      Core 0                 Core 1
  (Writes sum[0])        (Writes sum[1])
         |                      |
         +---- MESI Protocol ---+
         Cache Line Invalidated!
         Bounces across interconnect on EVERY write!
```

Podium Talking Point: *"Look at this whiteboard: Thread 0 and Thread 1 think they are independent because they write to different array indices `sum[0]` and `sum[1]`. But hardware caches do NOT move individual 4-byte integers; hardware caches move 64-byte cache lines! Because both indices live on the same cache line, Core 0 invalidates Core 1's cache on every iteration. You added parallel threads, yet your code slowed down by 8x. This is called False Sharing. To fix it, we use thread-private local accumulation variables or align with `alignas(64)`."*

Schematic 2.2: Distributed Memory & MPI Interconnect



Podium Talking Point: *"In OpenMP, threads share an address space: if Thread 0 writes a pointer, Thread 1 can read it directly. In MPI, Node 0 cannot touch Node 1's memory address space. Memory is completely private. If Node 0 needs Node 1's data, it MUST explicitly packetize, serialize, transmit across a network cable (InfiniBand/Ethernet), and deserialize. The network is your bottleneck."*

Day 3 & 4: Distributed AI Training & Profiling Roofline

Schematic 3.1: All-to-All vs Ring-AllReduce (Horovod / NCCL)

```
RING-ALLREDUCE TOPOLOGY:
Nodes arranged in a logical ring: [Node 0] -> [Node 1] -> [Node 2] -> [Node 3] -> [Node 0]

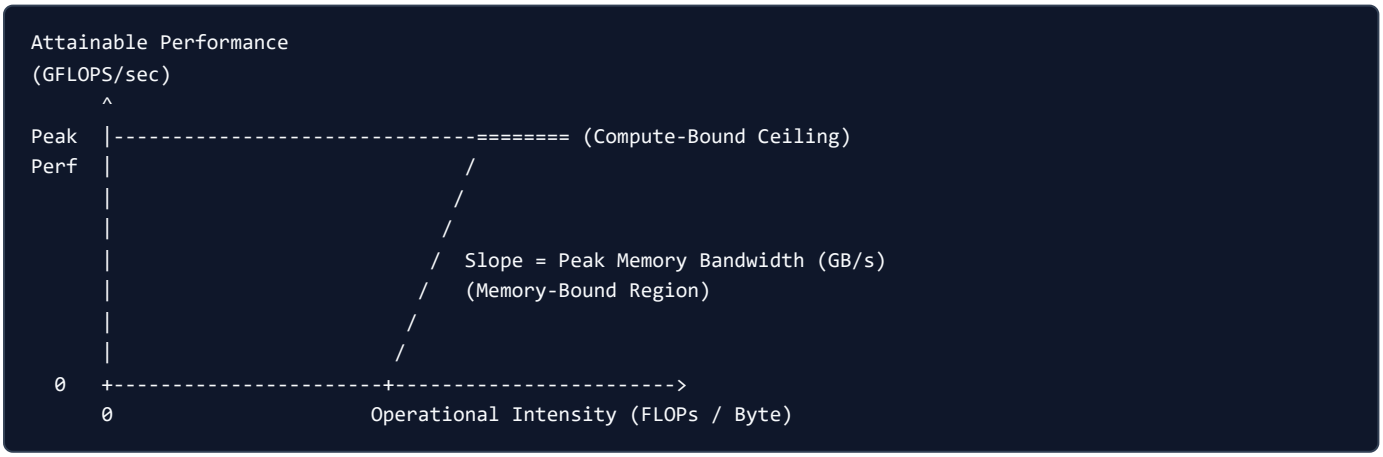
Phase 1: Scatter-Reduce (N-1 steps)
Each node sends 1/N of its gradient chunk to its neighbor while accumulating received chunks.

Phase 2: All-Gather (N-1 steps)
Each node sends the fully reduced chunk around the ring.

TOTAL DATA TRANSFERRED PER NODE = 2 * ((P - 1) / P) * BufferSize
Notice: Independent of the number of nodes P as P grows large!
```

Podium Talking Point: *"If you do naive All-to-All gradient syncing across 128 GPUs, communication overhead explodes as $O(P^2)$ and crashes the cluster switch. Ring-AllReduce breaks the gradient tensor into P chunks. Each worker communicates ONLY with its immediate left and right neighbor. The bandwidth per GPU remains constant regardless of whether you have 4 GPUs or 4,000 GPUs."*

Schematic 4.1: The Cache-Aware Roofline Model (VTune Profiling)



Podium Talking Point: *"When you run Intel VTune, it plots your program as a dot on this Roofline chart. If your dot falls on the diagonal slope, your code is Memory-Bound: your CPU ALU is starving for bytes from RAM. Adding faster cores will not help you; you must increase Operational Intensity (cache reuse, loop tiling, blocking). If your dot is under the horizontal ceiling, your code is Compute-Bound: you need vectorization (AVX-512) and instruction pipelining."*

Day 5: Generative AI, LLM Serving & Quantization

Schematic 5.1: LLM Inference Anatomy: Prefill vs Decode Phase

```
1. PREFILL PHASE (Prompt Processing):
   - Computes attention across all input tokens simultaneously.
   - GEMM (General Matrix-Matrix Multiply): Compute-Bound.
   - High GPU/CPU compute utilization.

2. DECODE PHASE (Autoregressive Token Generation):
   - Generates ONE token at a time: x_{t+1} = Model(x_t, KV_Cache).
   - GEMV (General Matrix-Vector Multiply): MEMORY-BOUND!
   - Every single parameter of the 7-Billion parameter model must be read
     from HBM/DRAM into registers JUST TO GENERATE A SINGLE TOKEN!
```

Podium Talking Point: *"Why does generating text feel slow? In the decode phase, to output one single word, you must stream all 14 Gigabytes of FP16 model weights through the memory bus! Memory bandwidth is the absolute bottleneck of generative AI inference. That is why INT8 and INT4 quantization gives a 2x-4x speedup: it cuts the memory bus traffic in half!"*

---

3. Whiteboard Execution Rules

1. **Always Cap Whiteboard Sessions at 15-20 Minutes**:
   - Never lecture continuously for 60 minutes.
   - Flow: 15 min whiteboard concept -> 10 min live terminal demo -> 35 min student hands-on lab.
2. **Use Contrasting Color Codes**:
   - Black: Hardware boundaries and compute units (ALU, Cores, Nodes).
   - Blue: Memory and data flow (Caches, DRAM, Network cables).
   - Red: Bottlenecks, invalidations, race conditions, cache misses.
   - Green: Optimized solution (AVX vectorization, Ring-AllReduce, INT8 quant).